Phase 2

Manya Jain **2024102068**

Architecture

Both models share the same architecture: a causal decoder-only Transformer built entirely from scratch. No third-party model code was used.

Multi-Head Causal Self-Attention

Attention is implemented using the standard formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

Here M is an upper-triangular mask filled with $-\infty$, so no token can attend to positions ahead of it. The reshaping between (B, T, d_{model}) and (B, n_h, T, d_k) is handled manually rather than relying on a library for it.

Block Structure

Each block uses **pre-LayerNorm**, meaning LayerNorm is applied before the sublayer rather than after. This made training noticeably more stable in practice. Both the attention and feed-forward sublayers have residual connections around them:

$$x \leftarrow x + \text{Attn}(\text{LN}(x)), \quad x \leftarrow x + \text{FFN}(\text{LN}(x))$$

GELU is used as the activation function for the feed-forward layer.

Weight Tying

The output projection matrix W_{out} is tied to the token embedding matrix W_{emb}, so they share the same weights. This cuts down the parameter count, and seems to help stability a little during training.

Configuration

Table 1: Architecture hyperparameters for both models.

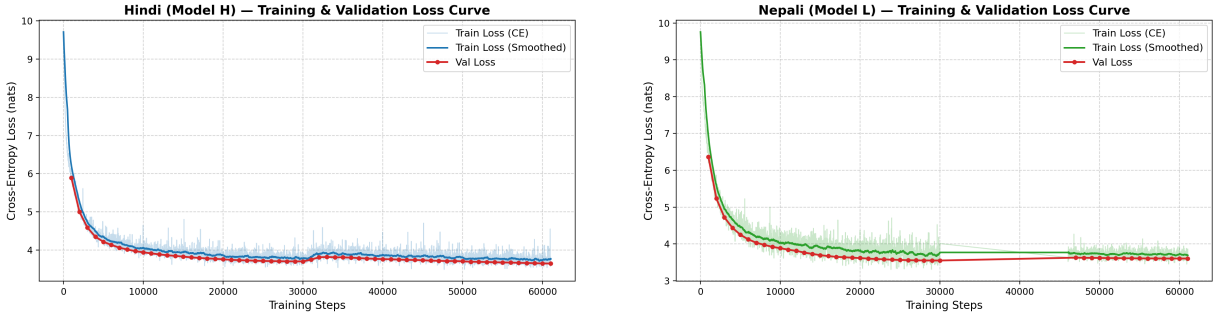
Parameter	Model H (Hindi)	Model L (Nepali)
Vocabulary size	16,000	16,240
Hidden dim d_{model}	512	512
Attention heads	8 ($d_k = 64$)	8 ($d_k = 64$)
Transformer layers	6	6
FFN dim d_{ff}	1,536 ($3d$)	1,536 ($3d$)
Context length	512 tokens	512 tokens
Dropout	0.1	0.1
Total parameters	**≈24.22M**	**≈24.34M**

Pretraining Details

Both models were pretrained with AdamW, using gradient clipping and a cosine learning rate schedule with linear warmup. Checkpoints were saved periodically with the full training state (weights, optimiser, and scheduler) so training could be resumed if something crashed.

- **Total steps:** 61,036 for each language.
- **Tokeniser:** Custom byte-pair encoding (BPE) tokenisers, vocabulary size 16K.
- **Data format:** Memory-mapped binary token arrays for efficient streaming.

Loss Curves



(a) Model H (Hindi) — training and validation loss. **(b)** Model L (Nepali) — training and validation loss.

Figure 1: Training and validation cross-entropy loss (nats) over 61,036 steps. Both models converge steadily with no obvious overfitting, and validation loss tracks the smoothed training curve closely throughout.

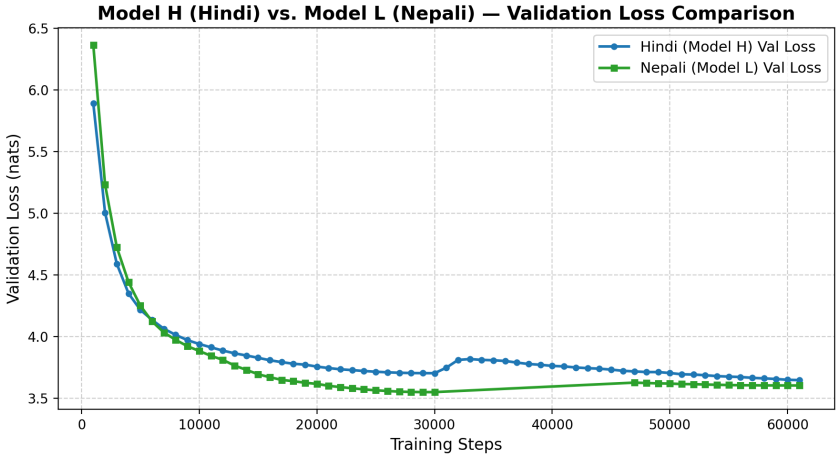


Figure 2: Validation loss comparison between Model H and Model L. Nepali ends up at a slightly lower validation loss, matching its lower final perplexity.

Intrinsic Evaluation: Perplexity and Bits-Per-Byte

Metrics

- **Cross-entropy loss** (nats): $\mathcal{L} = -\frac{1}{N}\sum_t \log p_\theta(x_t \mid x_{<t})$
- **Perplexity:** $\text{PPL} = e^{\mathcal{L}}$
- **Bits-per-byte (BPB):** $\text{BPB} = \frac{\mathcal{L}}{\ln 2 \cdot \bar{b}}$, where $\bar{b}$ is the mean UTF-8 bytes per token, normalising for tokeniser fertility across scripts.

Results

Table 2: Intrinsic evaluation results on the held-out test split.

Model	Test Loss (nats)	PPL	BPB	Avg Bytes/Token
Model H — Hindi	3.666	**39.11**	0.513	10.31
Model L — Nepali	3.592	**36.32**	0.394	13.16

Nepali comes out with both lower perplexity and noticeably lower BPB. Part of that is structural: the Nepali tokeniser produces longer tokens on average (13.2 bytes vs. 10.3 bytes), which compresses the corpus more and lowers the per-byte uncertainty. Both models look well converged – at checkpoint 61,000, validation loss is 3.649 for Hindi and 3.605 for Nepali.

Generation Quality Evaluation

Text continuations were generated from held-out prefix sequences using four decoding strategies: greedy argmax, and temperature sampling at $T \in \{0.5, 1.0, 1.5\}$.

Overlap Metrics

BLEU-4 and chrF(++) are computed using `sacrebleu`. ROUGE-L uses a whitespace tokeniser suited to Devanagari script, since the default character-level tokeniser in `rouge-score` doesn't segment Devanagari correctly at word boundaries.

Table 3: Generation quality metrics by decoding strategy.

Decoding	Model	BLEU-4	chrF	ROUGE-L
Greedy	Hindi (H)	0.95	12.21	0.124
	Nepali (L)	1.22	13.54	0.084
Temp 0.5	Hindi (H)	0.77	13.45	0.092
	Nepali (L)	1.56	19.88	0.097
Temp 1.0	Hindi (H)	0.72	12.71	0.091
	Nepali (L)	1.06	19.27	0.090
Temp 1.5	Hindi (H)	0.84	11.56	0.096
	Nepali (L)	0.14	16.08	0.039

The overlap scores are low to modest across both languages, but that's expected for open-ended continuation without any constrained decoding. BLEU-4 in particular gets penalised by short or mismatched references. chrF, since it works at the character level, does a better job of catching partial word overlap in morphologically rich languages like Hindi and Nepali.

Diversity and Repetition Diagnostics

Table 4: Lexical diversity metrics: repetition rate (4-gram), Distinct-1, and Distinct-2.

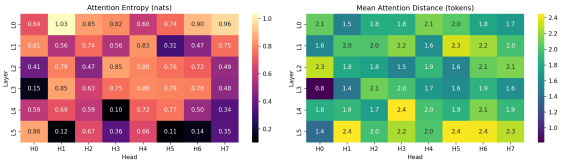
Decoding	Model	Rep. Rate (4-gram)	Distinct-1	Distinct-2
Greedy	Hindi (H)	0.266	0.427	0.622
	Nepali (L)	0.671	0.229	0.317
Temp 0.5	Hindi (H)	0.101	0.461	0.734
	Nepali (L)	0.335	0.398	0.628
Temp 1.0	Hindi (H)	0.027	0.684	0.931
	Nepali (L)	0.021	0.721	0.959
Temp 1.5	Hindi (H)	0.000	0.945	1.000
	Nepali (L)	0.000	0.908	0.999

There's a clear trade-off between temperature and diversity in both models. Greedy decoding falls into repetitive loops pretty easily (Nepali especially, with a repetition rate of 0.671), which is a known issue with deterministic decoding when the model isn't perfectly trained. $T = 0.5$ seems to be a good middle ground – repetition drops a lot while the text stays reasonably on-topic. By $T = 1.5$ repetition is basically gone, but the generations start to lose coherence.

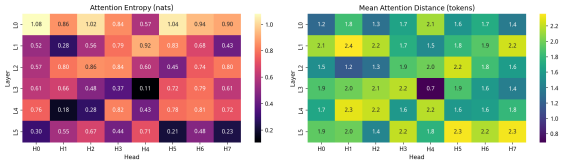
Multi-Head Attention Analysis

Attention weight distributions were extracted from three representative layers (Layer 0, Layer 3, Layer 5) and characterised using two statistics:

- **Attention entropy:** $H = -\sum_j \alpha_j \log \alpha_j$ — higher entropy means more diffuse attention.
- **Mean attention distance:** $\bar{d} = \sum_j \alpha_j |i - j|$ — measures average positional span attended to.

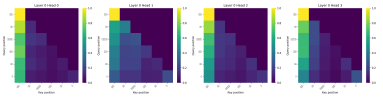


(a) Model H (Hindi): entropy and mean distance per layer/head.

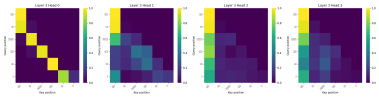


(b) Model L (Nepali): entropy and mean distance per layer/head.

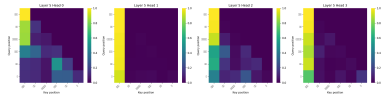
Figure 3: Per-layer, per-head attention entropy and mean attention distance. Heads clearly diverge in function as depth increases.



(a) Layer 0 (early).

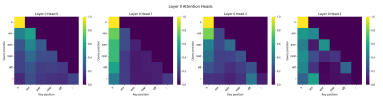


(b) Layer 3 (middle).

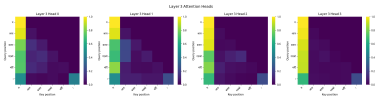


(c) Layer 5 (deep).

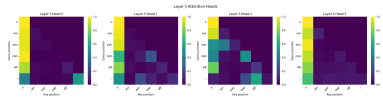
Figure 4: Attention heatmaps for Model H (Hindi) at three depths. Early layers focus locally along the diagonal, and deeper layers shift toward broader, non-local patterns.



(a) Layer 0 (early).



(b) Layer 3 (middle).



(c) Layer 5 (deep).

Figure 5: Attention heatmaps for Model L (Nepali). The same depth pattern shows up here too – early layers stay mostly local, deeper layers pull in broader context.

Observations

1. **Layer 0 (Early):** Attention stays strongly localised to nearby positions – there's a clear diagonal band pattern, with mean distance around 1.5–2.1. These heads are probably picking up immediate n-gram context and subword-level structure.
2. **Layer 3 (Intermediate):** Heads start to specialise here. Some stay locally focused, while others begin attending across phrase or clause boundaries, which suggests they're starting to pick up on syntactic structure.
3. **Layer 5 (Deep):** Entropy goes up and mean distance stretches out to about 2.0–2.5, with attention spread more diffusely over longer ranges. Right before the output projection, these heads seem to be pulling together discourse- and sentence-level context, which matches what's generally reported about deeper layers in Transformers.

Causal Masking Verification

The causal masking was verified programmatically: changing the final token in a sequence leaves all the earlier token logits unchanged to numerical precision (max logit diff $< 10^{-5}$). This confirms the causal mask correctly blocks information from flowing backward from future positions.

Nepali ends up with a lower perplexity than Hindi despite being the lower-resource language, which is worth explaining. The Nepali tokeniser produces about 28% more bytes per token on average (13.16 vs. 10.31), so each token is carrying more information. That compresses the corpus more, which probably makes the next-token prediction problem a bit easier relative to the effective sequence length. It's also possible Nepali has fewer orthographic ambiguities than Hindi, which has a broader vocabulary to disambiguate.

Both models also show a strong repetition problem under greedy decoding, which is pretty typical for autoregressive models trained only on negative log-likelihood, without any extra diversity objective like contrastive decoding or repetition penalties. Sampling at $T = 0.5$ fixes most of that while keeping the text reasonably on-topic. At moderate temperatures both models seem to have learned continuation patterns that are at least grammatically plausible.

In this phase, two monolingual decoder-only Transformer language models were built from scratch ($\approx$24M parameters each) and pretrained on Hindi and Nepali corpora. Both converge to reasonable perplexities (39.1 and 36.3), show the depth-dependent attention specialisation one would expect, and generate topically plausible text at moderate sampling temperatures. Of the temperatures tested, $T = 0.5$ gives the best trade-off between fluency and diversity. All the models, training logs, evaluation scripts, and generated artefacts are in the repository.